

Getting Started with STM32CubeIDE and SensorTile

This lab tutorial aims to guide you through the process of installing the programming environment for the used embedded device and building example projects which will demonstrate the use of the *XPRINTF* function and the provided Bluetooth interface to display data over USB on your laptop or over Bluetooth on a mobile app. While printing on laptop is not an essential part of the project, it can be useful for debugging and checking sensor data validity. **To begin this lab, download the example project provided on Canvas.** This project already contains the necessary abstraction layer you need for your project, so use this project as your starting point for this lab exercise and for the project.

Lab preparation

Before attempting this lab, you should install the following software and app, if needed, on your personal devices:

1. STM32CubeIDE: The software IDE for STM development kit, which can be downloaded from: <https://www.st.com/en/development-tools/stm32cubeide.html>
The CubeIDE is already installed in the MDLS lab machine. For your convenience, you may choose to install it in your own laptop. For Windows users, you may also need to install the virtual COM port driver to be able to use the XPRINTF function over USB: <https://www.st.com/en/development-tools/stsw-stm32102.html> (use the Windows 8 driver for Windows 10 or above).
2. CoolTerm: The terminal program for displaying the output from the XPRINTF function over USB. This can be downloaded from <http://freeware.the-meiers.org/>. This can be replaced by other terminal programs such as PuTTY.
3. ST BLE Sensor app: This is a mobile app that you can use to display data sent over the Bluetooth interface.
Android: <https://play.google.com/store/apps/details?id=com.st.bluems>
iOS: <https://apps.apple.com/it/app/st-bluems/id993670214>

Using the SensorTile Development Kit



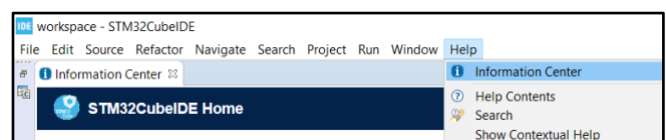
The kit supplied to you has the SensorTile, a STLINK-V3 programmer and a rechargeable battery, all integrated in one unit. The USB interface allows you to program the SensorTile and also view the debug messages sent using XPRINTF(). While plugged in, the USB interface also charges the on-board battery.

The battery can be used to run the device with the USB unplugged. With the USB port facing you, the switch turns the device ON (up position) and OFF (down position).

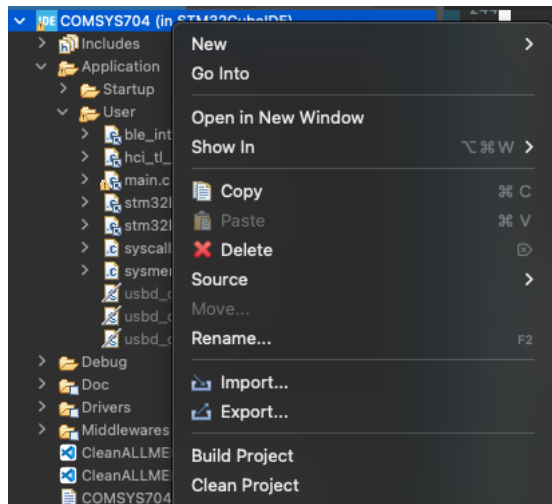
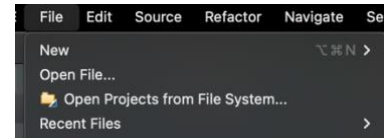
Note: When the USB is plugged in, the device is always on, irrespective of the switch position.

Implement, Build, and Run a xprintf example

Once the CubeIDE is installed, open the IDE and create your own workspace directory. Once the workspace has loaded, you should arrive at the **Information Centre** page, which is not needed and can be closed.



Download the provided template project (i.e. COMSYS704.zip) from Canvas and unzip it to the **workspace directory**. You can open the project in CubeIDE by choosing “**Open Projects from File System**” under the **File** menu. When importing the project, make sure you **only** tick the following project:



Once the project is loaded, under the project navigation pane, go to Application -> User -> main.c, double click to open the file. In the main() function, the following functions are already provided to initialise the SensorTile device for you. You can go through them if you are interested, but you don't need to change anything in those functions.

```
HAL_Init();

// Configure the System clock
SystemClock_Config();

InitTargetPlatform();

// Initialize the BlueNRG
Init_BlueNRG_Stack();

// Initialize the BlueNRG Custom services
Init_BlueNRG_Custom_Services();

// Initialize timers
InitTimers();
```

In the main() function, the **ReadSensor** flag is set every 100ms. In this exercise, the goal is to show how to use the XPRINTF function, so you can print a simple periodically incremented number example as follows:

```
if (ReadSensor) {
    ReadSensor=0; // reset flag

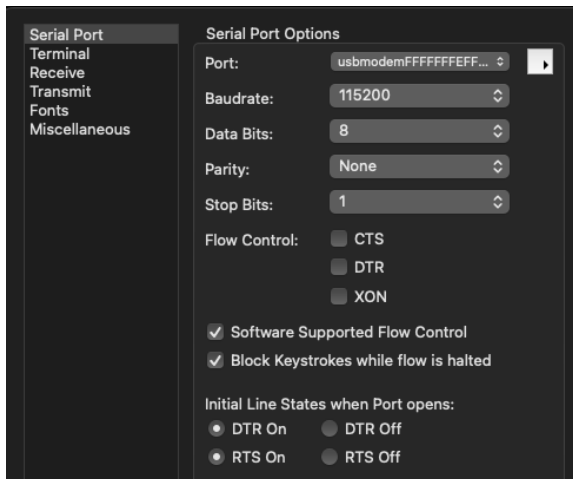
    //*****get sensor data*****
    readMag();
    readAcc();

    //*****process sensor data*****
    COMP_Value.Steps++;
    COMP_Value.Heading+=5;
    COMP_Value.Distance+=10;

    XPRINTF("Steps = %d \r\n", (int)COMP_Value.Steps);
}
```

The XPRINTF function works through the USB-UART interface via the USB port on your device, which can be displayed through a terminal program. Once complete a simple example, **build** , and then either **run**  the project by selecting the relevant button from the toolbar.

In this exercise, we are using CoolTerm as the terminal program. As mentioned earlier, you can use any terminal program that you are familiar with, with the following settings. Once the terminal program is connected, the XPRINTF statements will be displayed.



Implement, Build, and Run a Bluetooth communication example

In this example, you will modify the project to display data values on the ST BLE Sensor App via the provided Bluetooth communication. The following global variables are defined to store the raw Accelerometer and Magnetometer sensor data. The processed data **COMP_Value** variables are defined to store the detected number of Steps, Heading angle, and Distance. (**Note:** we will not use the “Distance” variable in this project, so you can utilise this variable for debugging purposes, or don’t use it.) These stored data (**ACC_Value**, **MAG_Value**, and **COMP_Value**) will be automatically sent over Bluetooth every 100ms by setting the **SendAccGyroMag** flag in the infinite event while loop.

ACC_Value.x, ACC_Value.y, ACC_Value.z &

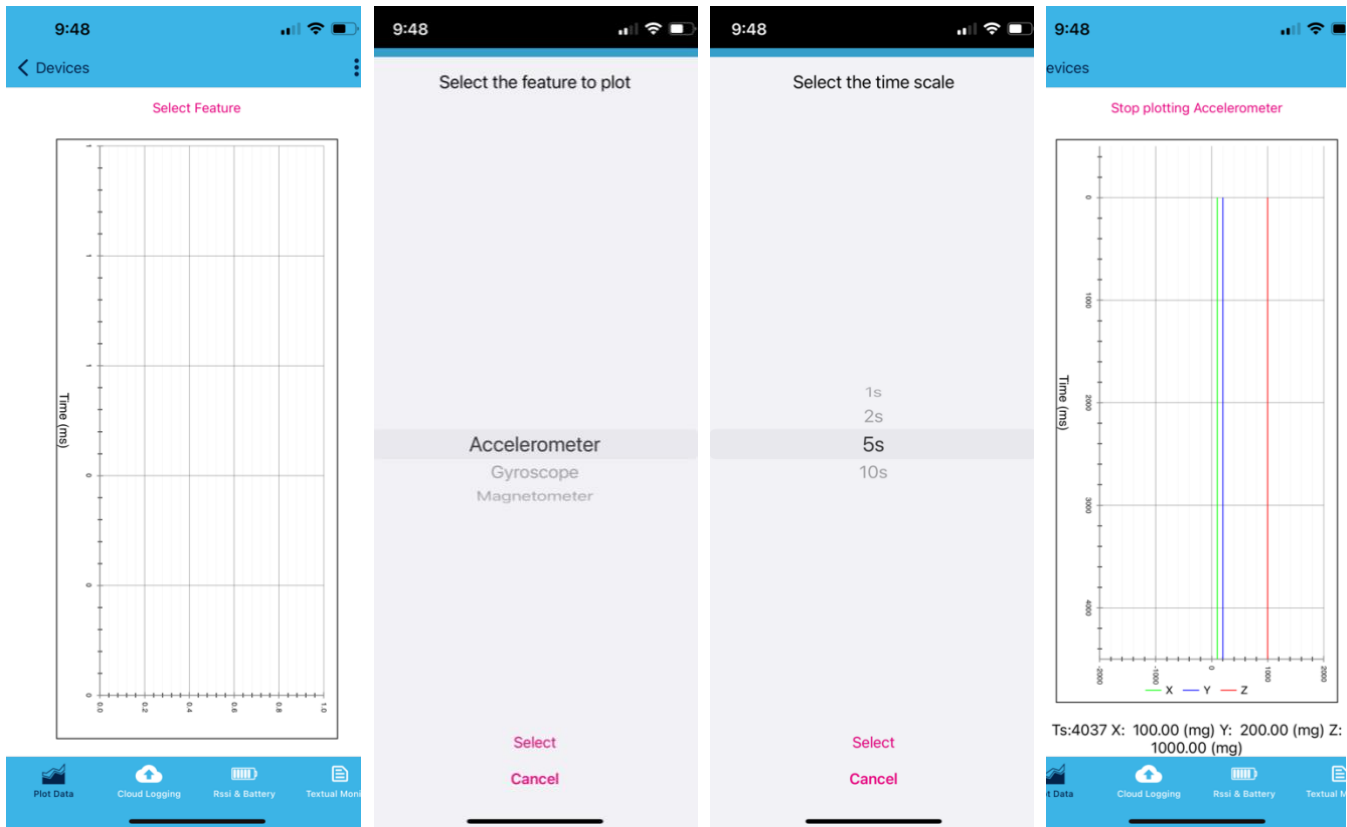
MAG_Value.x, MAG_Value.y, MAG_Value.z &

COMP_Value.Steps , COMP_Value.Heading, COMP_Value.Distance

In addition to the raw sensor and processed data value, the device name can be changed through the **NodeName[8]** variable. To emulate the raw sensor and processed data, you can simply try incrementing the **ACC_Value**, **MAG_Value**, and **COMP_Value** variables. once the code is complete, **build** , and then **run**  the project by selecting the relevant button from the toolbar.

Once you started running the project, open the ST BLE Sensor app and select “Connect to a device”. There should be a device with the same name defined by the **NodeName** variable showing up as in the figure below. Click on the device to select it. The data received by the mobile BLE app will be plotted separately by selecting a **feature and time scale** (i.e. Accelerometer (**ACC_Value**), Magnetometer (**MAG_Value**), or Gyroscope (**COMP_Value**)) respectively as shown in the figures next page.





Once you are done with the lab, it is time for you to dig in into the datasheet to understand how to configure the LSM303AGR (the integrated accelerometer and magnetometer sensor). The SPI bus interface (i.e. the SPI read and write) functions are provided to you. You will need to initialise the sensor and apply the necessary compensation/calibration (if any), and figure out how to interpret the sensor data such that it can be used to implement your algorithm (with step counting and heading angle detection).

Implement the following functions to initialise and read data from the Magnetometer and Accelerometer.

```
static void startMag() {
    //CS704 - Write SPI commands to initiliasie Magnetometer
}

static void startAcc() {
    //CS704 - Write SPI commands to initiliasie Accelerometer
}

static void readMag() {

    //CS704 - Read Magnetometer Data over SPI

    //CS704 - store sensor values into the variables below
    MAG_Value.x=100;
    MAG_Value.y=200;
    MAG_Value.z=1000;
}

static void readAcc() {

    //CS704 - Read Accelerometer Data over SPI

    //CS704 - store sensor values into the variables below
    ACC_Value.x=100;
    ACC_Value.y=200;
    ACC_Value.z=1000;
}
```